Beyond Dynamic Programming

State spaces, speed-ups, and Bellman’s equations

Your Name

Workshop

Your Affiliation

Outline

1. Dynamic Programming from First Principles
2. Is It Dynamic Programming? Case Studies
3. Going Faster
4. Beyond Programming Problems: MDPs
5. Other Places, and What We Learned

Dynamic Programming from First Principles

Calibration

This workshop assumes strong programming and discrete mathematics.
Recurrences are written mathematically, never as code.

Audience | Poll (show of hands)

1. Have you implemented the convex hull trick or a Li Chao tree?
2. Can you state the Bellman optimality equation from memory?
3. Is Dijkstra's algorithm a dynamic program?

A motivating problem: minimum edit script

Version control. Given two versions of a file as token sequences $x \in \Sigma^m$ and $y \in \Sigma^n$, produce a cheapest sequence of edits that turns x into y.

- **delete** a token of x — cost 1
- **insert** a token of y — cost 1
- **substitute** a token — cost 1 (cost 0 if the tokens already agree)

Goal: compute $\mathrm{ED}(x, y)$, the minimum total cost, and an optimal script. A script is a sequence of $\leq m + n$ local decisions: the number of scripts is exponential. What is a good way to organise the search?

State spaces

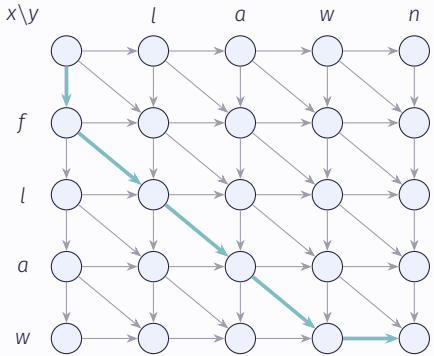
Definition. A state space is a weighted digraph $G = (V, E, w)$ with source s and target t: solutions of the problem correspond to $s \to t$ paths, and the objective is the sum of edge weights.

- A state v summarises a *partial solution*.
- Sufficiency (Markov property): the feasible completions of v, and their costs, depend only on v, not on the path that reached it.

Edit distance. State (i, j): "$x_{1..i}$ has been transformed into $y_{1..j}$". Source $(0, 0)$, target (m, n).

$(i-1, j) \to (i, j)$	delete x_i	$w = 1$
$(i, j-1) \to (i, j)$	insert y_j	$w = 1$
$(i-1, j-1) \to (i, j)$	match / substitute	$w = \mathbb{1}[x_i \neq y_j]$

The state space is a DAG



- Every edge increases $i + j$: acyclic.
- Topological order: row-major (any order by $i + j$).
- Highlighted: `flaw` $\rightarrow$ `lawn`, cost 2 (delete `f`, three matches, insert `n`).

$(m + 1)(n + 1)$ states, ≤ 3 in-edges each.

Paths versus states

The number of $s \to t$ paths in the $n \times n$ grid with the three step types is the central Delannoy number $D(n, n) \sim c\,(3 + 2\sqrt{2})^n$, while there are only $(n+1)^2$ states.

Audience | Poll

For $m = n = 20$: roughly how many edit scripts (paths) are there?

(a) 10^6 (b) 10^9 (c) 10^{12} (d) $> 10^{14}$

Paths versus states

The number of $s \to t$ paths in the $n \times n$ grid with the three step types is the central Delannoy number $D(n, n) \sim c\,(3 + 2\sqrt{2})^n$, while there are only $(n+1)^2$ states.

Audience | Poll

For $m = n = 20$: roughly how many edit scripts (paths) are there?

(a) 10^6 (b) 10^9 (c) 10^{12} (d) $> 10^{14}$

$$D(20, 20) = 260{,}543{,}813{,}797{,}441 \approx 2.6 \cdot 10^{14} \text{ paths} \quad \text{versus} \quad 441 \text{ states.}$$

Dynamic programming exists because many paths share the same sub-paths: the state graph is a compressed representation of the solution space.

Dynamic programming on a DAG

Theorem. Let $G = (V, E, w)$ be a DAG with source s, and $d(v)$ the minimum weight of an $s \to v$ path (∞ if none). Then

$$d(s) = 0, \qquad d(v) = \min_{(u,v)\in E} \{\, d(u) + w(u, v) \,\} \quad (v \neq s).$$

Proof sketch. An $s \to v$ path is an $s \to u$ path followed by an edge (u, v) and its weight is the sum of the two parts. The choice of prefix does not constrain the last edge, so minimising each prefix is lossless (optimal substructure). Acyclicity makes the equations well-founded: induct along a topological order.

Cyclic graphs satisfy the same equations, but they no longer *define* d by induction (Part 4).

A strategy for dynamic programming problems

1. **Decompose.** Choose a state space: what must be remembered about a partial solution so that the future is independent of the past?
2. **Recurrence.** State the base case and the inductive step expressing each state through states that precede it.
3. **Evaluate bottom-up** in a topological order of the state DAG.
4. **Reconstruct** (if the solution, not only its value, is needed) by recording or recomputing the minimising predecessor of each state.

Complexity is read off the DAG:

$$T \;=\; \sum_{v \in V} \big(\text{work to evaluate state } v\big) \;=\; |V| \times (\text{cost per state}) \;\approx\; |E|.$$

Steps 1–2: the recurrence for edit distance

Let $f(i, j)$ be the minimum cost of transforming $x_{1..i}$ into $y_{1..j}$.

$$\text{base:}\quad f(i,0) = i, \qquad f(0,j) = j$$

$$\text{step:}\quad f(i,j) = \min \begin{cases} f(i-1,j) + 1 & \text{delete } x_i \\ f(i,j-1) + 1 & \text{insert } y_j \\ f(i-1,j-1) + \mathbb{1}\left[x_i \neq y_j\right] & \text{match / substitute} \end{cases}$$

Answer: $\mathrm{ED}(x, y) = f(m, n)$.

Why correct? Consider the *last* operation of an optimal script for $(x_{1..i}, y_{1..j})$. It is one of three kinds, and the script before it is a script for the corresponding smaller pair; it must itself be optimal, otherwise it could be replaced by a cheaper one.

Step 3: bottom-up evaluation

Evaluate $f(i, j)$ row by row (a topological order). Rows: `flaw`; columns: `lawn`.

Audience | Predict

Fill in the last two rows before we reveal them.

	ε	l	a	w	n
ε	0	1	2	3	4
f	1	1	2	3	4
l	2	1	2	3	4
a	?	?	?	?	?
w	?	?	?	?	?

Step 3: bottom-up evaluation

Evaluate $f(i, j)$ row by row (a topological order). Rows: `flaw`; columns: `lawn`.

Audience | Predict

Fill in the last two rows before we reveal them.

	ε	l	a	w	n
ε	0	1	2	3	4
f	1	1	2	3	4
l	2	1	2	3	4
a	3	2	1	2	3
w	4	3	2	1	2

Step 4: reconstruction

Store (or recompute) the minimising predecessor of each state; walk back from (m, n):

$$(4,4) \xleftarrow{\text{insert n}} (4,3) \xleftarrow{\text{match w}} (3,2) \xleftarrow{\text{match a}} (2,1) \xleftarrow{\text{match l}} (1,0) \xleftarrow{\text{delete f}} (0,0).$$

f	l	a	w	-
-	l	a	w	n
del	=	=	=	ins

- Recomputing the argmin on the way back needs only the value table.
- The value table costs $\Theta(mn)$ memory. Rolling two rows gives $\Theta(\min(m, n))$ but loses reconstruction; Hirschberg's divide-and-conquer restores it at twice the time and linear space.

Complexity and its limits

$|V| = (m+1)(n+1)$, at most 3 predecessors each: $\Theta(mn)$ time.

- **Output-sensitive.** If $\mathrm{ED} \le d$, only the band $|i-j| \le d$ matters: $O(d \cdot \min(m,n))$ (Ukkonen); Myers' $O(nd)$ diff is the engine behind `git diff`.
- **Lower bound.** No $O(n^{2-\varepsilon})$ algorithm exists unless SETH fails (Backurs–Indyk, STOC 2015).

Lesson: *the state space fixes a baseline*; structure of the instance or of the cost function can beat it, and conditional lower bounds tell us when to stop trying.

Is the state space really sufficient?

Audience | Discuss

Allow **transpositions** of adjacent tokens ($ab \to ba$) at cost 1. (1) Which edge do you add to the DAG, and when? (2) Does the complexity change? (3) Is the result exactly the Damerau–Levenshtein distance?

Is the state space really sufficient?

Audience | Discuss

Allow **transpositions** of adjacent tokens ($ab \to ba$) at cost 1. (1) Which edge do you add to the DAG, and when? (2) Does the complexity change? (3) Is the result exactly the Damerau–Levenshtein distance?

1. Add $(i-2, j-2) \to (i, j)$ with cost 1 when $x_i = y_{j-1}$ and $x_{i-1} = y_j$.
2. Still $\Theta(mn)$: one more $O(1)$ in-edge.
3. No. This is the *optimal string alignment* distance (no substring is edited twice). For $x =$ CA, $y =$ ABC it gives 3, the true distance is 2 (CA→AC→ABC). Our edges are not sufficient: the exact algorithm needs edges reaching back to the last occurrence of a symbol (see backup).

Part 1 in one slide

What made this a dynamic program?

1. **A sufficient state**, giving a graph that compresses exponentially many solutions.
2. **Acyclicity**, giving a well-founded order of evaluation.
3. **Additive (or otherwise decomposable) objective**, giving optimal substructure.

Design is in step 1. Steps 2–4 are mechanical once the state space is right, and so is the complexity: $|V|\times$ cost per state.

Next: problems where the recipe looks applicable. Does it help?

Is It Dynamic Programming? Case Studies

How this part works

For each problem you will see candidate solutions. Your job is to decide:

1. Is it **correct**?
2. Is it **the right tool**?

Triage test for dynamic programming

1. Is there a *sufficient* state of polynomial size? (Markov property)
2. Does the objective decompose over it? (optimal substructure)
3. Is there no cheaper algorithm that exploits more structure?

A positive answer to (1) and (2) makes DP *correct*; (3) decides whether it is *good*.

Case 1: Split Array Largest Sum (LeetCode 410)

Given $a_1, \ldots, a_n \geq 0$ and $k \leq n$, split the array into k non-empty contiguous parts so as to minimise the largest part sum. Let $S_i = \sum_{l \leq i} a_l$.

Example. $a = (7, 2, 5, 10, 8)$, $k = 2$: the parts $(7, 2, 5) \mid (10, 8)$ give $\max\{14, 18\} = 18$, which is optimal.

We will look at two proposals.

Case 1: candidate solutions

Proposal A (recurrence)

Let $f(i, j)$ be the best value for the first i elements in j parts.

$$f(0,0) = 0, \quad f(i,0) = f(0,j) = \infty$$
$$f(i,j) = \min_{j-1 \le p < i} \max\{f(p, j-1),\ S_i - S_p\}$$

Answer $f(n, k)$.

Proposal B (greedy)

Let $\bar{S} = \lceil S_n / k \rceil$. Sweep left to right and close the current part as soon as adding the next element would exceed $\bar{S}$ (until $k - 1$ parts are closed).

Audience | Vote

For each proposal: **correct** or **incorrect**? And if correct, is it **efficient**?

Case 1: verdict

A is correct. The state (i, j) is sufficient (only the prefix length and number of parts matter), and $(\max, \min)$ composes. But the cost:

$$\underbrace{nk}_{\text{states}} \times \underbrace{O(n)}_{\text{per state}} = \Theta(kn^2).$$

B is wrong. Take $a = (1, 3, 2)$, $k = 2$: $\bar{S} = 3$, so B closes (1) and returns $(1) \mid (3, 2)$ with value 5, while $(1, 3) \mid (2)$ achieves 4.

Greedy heuristics that “balance” parts fail because the objective is the *maximum*, not a deviation from the mean. Is there a correct greedy at all?

Case 1: a better algorithm, and why DP is wasteful

Decide, then optimise. For a cap L let $c(L)$ be the minimum number of parts with every sum $\leq L$. The greedy "extend each part as far as possible" computes $c(L)$ in $O(n)$ (it stays ahead of any other partition).

- c is non-increasing in L; since $k \leq n$, a partition into fewer parts can always be refined.
- The answer is $\min\{L \in [\max_i a_i, S_n] \ : \ c(L) \leq k\}$, found by binary search.

Method	Time
Proposal A	$\Theta(kn^2)$
A with monotone optimum (binary search over p)	$O(kn \log n)$
Binary search on the answer + greedy	$O(n \log S_n)$

DP computes $f(i, j)$ for *every* prefix and part count, but we need one number.

Case 1: discussion

Audience | Discuss

1. Which property of “minimise the *maximum*” made the decision version greedy-checkable and monotone in L?
2. Suppose we instead minimise the **sum of squares** of part sums. Does binary search on the answer still work? What would you use?

Case 1: discussion

Audience | Discuss

1. Which property of "minimise the *maximum*" made the decision version greedy-checkable and monotone in L?
2. Suppose we instead minimise the **sum of squares** of part sums. Does binary search on the answer still work? What would you use?

1. The max commutes with refinement: splitting a part never increases the objective, and feasibility under a cap is local.
2. No: there is no threshold structure. This is a genuine DP, $f(i,j) = \min_p f(p, j-1) + (S_i - S_p)^2$, and its transition has the form that the convex hull trick exploits in Part 3.

Case 2: Maximum weight independent set on a tree

LeetCode 337 (*House Robber III*); the path case is LeetCode 198.

Tree $T = (V, E)$ rooted at r, weights $w_v \geq 0$. Choose an independent set I maximising $\sum_{v \in I} w_v$. Write $\mathrm{ch}(v)$ for the children and $\mathrm{gc}(v)$ for the grandchildren of v.

- For general graphs the problem is NP-hard.
- On a tree, the subtree of v is separated from the rest of the tree by the single vertex v.

Where would you put the state?

Case 2: candidate solutions

Proposal A Greedy: repeatedly take the heaviest remaining vertex and delete its neighbours.

Proposal B $f(v) = \max\{\, w_v + \sum_{c \in \mathrm{ch}(v)} f(c),\ \sum_{c \in \mathrm{ch}(v)} f(c) \}$

Proposal C $f(v) = \max\{\, w_v + \sum_{g \in \mathrm{gc}(v)} f(g),\ \sum_{c \in \mathrm{ch}(v)} f(c) \}$

Proposal D $f(v,1) = w_v + \sum_c f(c,0)$ and $f(v,0) = \sum_c \max\{f(c,0), f(c,1)\}$; answer $\max_b f(r,b)$.

Audience | Vote

Which proposals are **correct**? Look for a counterexample for those you reject.

Case 2: verdict

A and B are wrong. Path $2 - 3 - 2$:

- A takes the centre: value 3; optimum 4 (both ends).
- B, rooted at the centre: $f(\text{leaf}) = 2$, so $f(\text{root}) = \max\{3 + 4, 4\} = 7$. The term $w_v + \sum_c f(c)$ forgets that $f(c)$ may itself use c.

C and D are correct. In D the state records whether v is chosen:

$$f(v, 1) = w_v + \sum_c f(c, 0)$$

$$f(v, 0) = \sum_c \max_b f(c, b)$$

C is D with the state projected out: $f(v) = \max_b f(v, b)$.

Time $\Theta(\sum_v \deg v) = \Theta(n)$. Both formulations have the same state DAG (the rooted tree), but D carries $2n$ states and has *local* transitions. Enlarging the state to make the transition local is a standard trick.

Case 2: why trees, and what generalises

The tree makes v a separator: the two sides interact only through v's status.

Theorem (folklore)
Given a tree decomposition of width ω of an n-vertex graph, a maximum weight independent set can be computed in $O(2^{\omega}\,\omega\,n)$ time.

The state of a bag is the independent subset chosen inside it ($\leq 2^{\omega+1}$ states); the structure of the graph *is* the shape of the state DAG.

Same recipe, different shape. Trees ($\omega = 1$), cycles ($\omega = 2$), series-parallel graphs ($\omega \leq 2$), grids $k \times n$ ($\omega = k$).

Case 2: discussion

Audience | Discuss

Now let the graph be a **cycle** C_n with weights $w_1, \ldots, w_n$. The tree recurrence is no longer valid. Fix it, and give the complexity.

Case 2: discussion

Audience | Discuss

Now let the graph be a **cycle** C_n with weights $w_1, \ldots, w_n$. The tree recurrence is no longer valid. Fix it, and give the complexity.

Fix the status b_1 of vertex 1 and run the path DP on $1, \ldots, n$ with that boundary condition (for $b_1 = 1$ vertex n is forbidden): two path DPs, $\Theta(n)$ in total. Equivalently, enlarge the state to $(b_1, b_{\text{current}})$: this is the width-$2$ tree decomposition made explicit, with 4 states per position.

Case 3: Minimum Initial Energy to Finish Tasks (LeetCode 1665)

Tasks $i = 1, \ldots, n$ with (a_i, m_i), $a_i \le m_i$: to *start* task i you need at least m_i energy, and finishing it consumes a_i. Choose the order π and the initial energy E to finish all tasks, minimising E.

For an order π:

$$E(\pi) = \max_{t} \Big(m_{\pi(t)} + \sum_{s<t} a_{\pi(s)} \Big), \qquad E^\star = \min_{\pi} E(\pi).$$

The order is a free choice. There are $n!$ orders.

It looks like a perfect candidate for DP: a sequence of decisions, additive consumption.

Case 3: candidate solutions

Let $A(S) = \sum_{i \in S} a_i$.

Proposal A Sort by m_i in *decreasing* order and do tasks in that order.

Proposal B Sort by a_i in *increasing* order.

Proposal C $F(\emptyset) = 0,\ F(S) = \min_{i \in S} \max\{F(S \setminus \{i\}),\ A(S \setminus \{i\}) + m_i\}$; answer $F([n])$.

Audience | Vote

Correct or incorrect? Is C a “dynamic programming solution”? Is it a *useful* one?

Case 3: verdict

Tasks written $[a_i, m_i]$.

- **A is wrong:** $[1,2], [3,3]$. A runs $[3,3]$ first and needs $\max\{3,\ 3+2\} = 5$; running $[1,2]$ first needs $\max\{2,\ 1+3\} = 4$.
- **B is wrong:** $[1,1], [2,3]$. B needs $\max\{1, 1+3\} = 4$; the other order needs $\max\{3, 2+1\} = 3$.
- **C is correct.** The energy left after finishing a set S is $E - A(S)$ *whatever the order*, so the set is a sufficient state.

But the state is the whole subset: 2^n states, $\Theta(2^n n)$ time. No polynomial-size sufficient state is apparent: the future depends on *which* tasks remain, not on a number summarising them. This is the sense in which the DP is a false positive: the formulation is valid, but the state does not compress.

Case 3: what actually solves it

Key $\kappa_i = m_i - a_i$. Claim: an order sorted by decreasing κ is optimal.

Exchange argument.

Suppose i is run immediately before j with $\kappa_i \geq \kappa_j$ versus the swapped order, with identical energy before and after the pair. Then

$$\begin{aligned} E(i,j) &= \max\{a_i + \kappa_i,\ a_i + a_j + \kappa_j\} \\ &\leq a_i + a_j + \kappa_i \leq \max\{m_j,\ a_j + m_i\} = E(j,i). \end{aligned}$$

Any order with an adjacent pair violating $\kappa_i \geq \kappa_j$ is therefore no better than the order with the pair swapped; repeated swaps sort the sequence (bubble sort) without increasing the requirement. □

Time $O(n \log n)$. The structure that makes this problem easy is an exchange argument, not a state space.

Rapid fire: DP, or not?

Audience | Vote

For each: **DP**, **something better**, or **no compact DP**?

1. Minimum number of coins summing to V.
2. Unweighted interval scheduling (maximum number of disjoint intervals).
3. Longest *simple* path in a general directed graph.
4. $0/1$ knapsack with $n = 40$ items and weights up to 10^{12}.
5. Longest increasing subsequence.
6. Shortest paths in a graph with cycles and non-negative weights.

Rapid fire: answers

1. **DP.** State $=$ remaining amount.
2. **Something better.** Earliest-finish-time greedy ($O(n \log n)$); DP is valid but unnecessary.
3. **No compact DP.** NP-hard; subset DP is $\Theta(2^n n)$, the state has to record the visited set.
4. **Something better.** $O(nW)$ is hopeless; meet-in-the-middle gives $O(2^{n/2}\, n)$.
5. **DP, then better.** $O(n^2)$ DP; changing the state to *(length $\to$ smallest tail)* gives $O(n \log n)$ by patience sorting.
6. **DP in disguise.** The Bellman equations hold but are cyclic; Dijkstra chooses an evaluation order. This is the theme of Part 4.

Going Faster

Where does the time go?

Time $\approx \sum_v \text{indeg}(v) = |E|$. Two levers, both about the *shape* of the DAG, not its meaning:

I **Change the recurrence.** Algebra on the transition turns "minimum over all predecessors" into a structured query (a data-structure operation).

II **Change the compute order.** Refine the DAG with intermediate states so that every state has $O(1)$ in-edges, computing the same values.

We illustrate (I) with the **convex hull trick** and (II) with a **bitmask profile DP**.

Lever I: a problem with a quadratic transition

Batched jobs. Jobs $1, \dots, n$ must be processed in order; job l has size $c_l \geq 0$ and $S_i = \sum_{l \leq i} c_l$. Partition the sequence into consecutive batches. A batch of total size s costs $M + s^2$ (set-up plus a quadratic penalty).

Let $\mathrm{dp}(i)$ be the optimal cost for the first i jobs:

$$\mathrm{dp}(0) = 0, \qquad \mathrm{dp}(i) = \min_{0 \leq j < i} \Big\{ \mathrm{dp}(j) + (S_i - S_j)^2 + M \Big\}.$$

n states, $\Theta(n)$ predecessors each: $\Theta(n^2)$. For $n = 10^6$ that is hopeless.

Audience | Discuss

Expand the square. **Which terms couple i and j?** What does that suggest about the inner minimisation?

Rewriting the recurrence

$$\mathrm{dp}(i) = S_i^2 + M + \min_{j<i} \Big\{ \underbrace{(-2S_j)}_{m_j} \cdot S_i + \underbrace{\mathrm{dp}(j) + S_j^2}_{b_j} \Big\}.$$

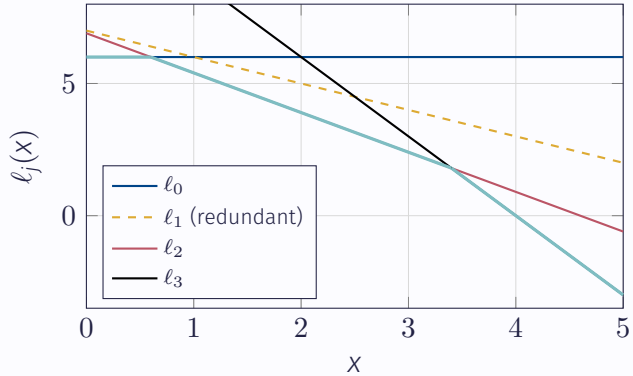
Only the cross term $-2S_iS_j$ couples i and j. Everything else is a function of i alone or of j alone.

Define the line $\ell_j(x) = m_j x + b_j$. Then

$$\boxed{\mathrm{dp}(i) = S_i^2 + M + \min_{j<i} \ell_j(S_i)}$$

The inner term is the lower envelope of the lines $\ell_0, \ldots, \ell_{i-1}$ evaluated at S_i. The line ℓ_i becomes available only after $\mathrm{dp}(i)$ is known (an *online* insertion).

The lower envelope



Slopes decrease with insertion order ($m_j = -2S_j$, S_j non-decreasing), so each new line is the best one for *large* x. The dashed line never attains the minimum and may be discarded.

The monotone convex hull trick

Keep the envelope as a deque Q of lines in insertion order; queries S_i are non-decreasing.

1. **Query.** While $|Q| \geq 2$ and $\ell_{Q_2}(S_i) \leq \ell_{Q_1}(S_i)$, pop the front. Then $\mathrm{dp}(i) = S_i^2 + M + \ell_{Q_1}(S_i)$.
2. **Insert** ℓ_i. While $|Q| \geq 2$ and the last line ℓ_2 (predecessor ℓ_1; slopes $m_1 > m_2 > m_i$) is redundant, pop the back:

$$\ell_2 \text{ redundant} \iff (b_i - b_1)(m_1 - m_2) \leq (b_2 - b_1)(m_1 - m_i),$$

i.e. ℓ_1, ℓ_i intersect left of ℓ_1, ℓ_2.

Each line is pushed once and popped at most once: amortised $O(1)$ per state, $\Theta(n)$ in total. The front can be discarded because later queries are larger and later lines have smaller slopes.

Robustness of the trick

Audience | Vote

The data are **not monotone**. Choose the data structure. (a) Slopes inserted in decreasing order, *arbitrary* query points. (b) *Arbitrary* slopes (e.g. some $c_l < 0$), arbitrary queries, online.

Robustness of the trick

Audience | Vote

The data are **not monotone**. Choose the data structure. (a) Slopes inserted in decreasing order, *arbitrary* query points. (b) *Arbitrary* slopes (e.g. some $c_l < 0$), arbitrary queries, online.

(a) Keep the hull as an array and binary-search the query: $O(n \log n)$ total.

(b) A Li Chao tree over the query domain (or a dynamic hull): $O(\log C)$ per insertion and query (C = size of the query domain; compress coordinates offline if needed).

Implementation note: the redundancy test multiplies differences of intercepts and slopes; for $S \sim 10^9$ the products need 128-bit arithmetic.

What structure was exploited?

The trick applies whenever the transition separates as $\mathrm{dp}(i) = \alpha_i + \min_{j<i}\{\beta_j + \gamma_i\,\delta_j\}$. Other structure in $\mathrm{cost}(j, i)$ gives other accelerations:

Structure of $\mathrm{cost}(j, i)$	Technique	Gain
Separable (inner product)	Convex hull trick / Li Chao tree	$n^2 \to n,\ n \log n$
Monge, 1D/1D online	Monotone optimum; SMAWK, LARSCH	$n^2 \to n \log n$ or n
Monge, k layers	Divide-and-conquer optimisation	$kn^2 \to kn \log n$
Convex in number of layers k	Lagrangian relaxation (“Aliens trick”)	removes factor k
Interval DP, monotone optimum	Knuth–Yao	$n^3 \to n^2$

Lever II: tilings and bitmask DP

Floorplanning. Count the tilings of an $n \times m$ board (some cells may be blocked) by 1×2 dominoes, with m small (≤ 16) and n large.

Row-by-row. Let $G_r(M')$ count partial tilings in which rows $< r$ are fully covered and $M' \subseteq \{0, \ldots, m-1\}$ are exactly the cells of row r covered by vertical dominoes from row $r-1$.

$$G_0(\emptyset) = 1, \qquad G_{r+1}(M') = \sum_{M \text{ compatible with } M'} G_r(M), \qquad \text{answer } G_n(\emptyset).$$

M and M' are compatible iff $M \cap M' = \emptyset$ and the cells of the row outside $M \cup M'$ split into runs of even length (to be tiled by horizontal dominoes).

2^m states per row. Compatible pairs: up to 3^m. Time $\Theta(n\, 3^m)$.

The edge count is the bottleneck

Audience | Discuss

There are only 2^m states per row, but 3^m edges between consecutive rows. Can the **same** counts be computed with fewer edges? *Hint:* what if we decide one cell at a time?

The edge count is the bottleneck

Audience | Discuss

There are only 2^m states per row, but 3^m edges between consecutive rows. Can the **same** counts be computed with fewer edges? *Hint:* what if we decide one cell at a time?

Insert intermediate states between rows: process cells in row-major order and keep, as state, the profile of the next m cells (already covered or not). Each cell has $O(1)$ choices: skip, place a vertical domino, place a horizontal domino. Same sums, regrouped; edges per state drop from 2^m to $O(1)$.

Broken profile DP

Number cells $t = rm + c$ in row-major order. Let $F_t(M)$, $M \in \{0,1\}^m$, count partial tilings in which cells $< t$ are covered, and bit k of M says whether cell $t + k$ is already covered.

$$F_0(0) = 1, \qquad \text{answer } F_{nm}(0).$$

From $F_t(M)$, with cell $t = (r, c)$:

$$M_0 = 1 : F_{t+1}\big(M \gg 1\big) \mathrel{+}= F_t(M)$$

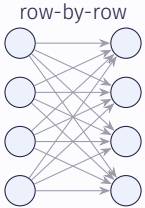
$$M_0 = 0 : F_{t+1}\big((M \gg 1) \mid 2^{m-1}\big) \mathrel{+}= F_t(M) \qquad \text{vertical, if } r + 1 < n$$

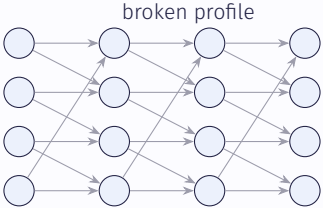
$$M_0 = M_1 = 0 : F_{t+1}\big((M \gg 1) \mid 1\big) \mathrel{+}= F_t(M) \qquad \text{horizontal, if } c + 1 < m$$

Blocked cells: forbid dominoes that would cover them and let a blocked cell pass through unchanged.

States: $nm\,2^m$. In-edges: ≤ 3. Time $\Theta(nm\,2^m)$ versus $\Theta(n\,3^m)$.

Same values, different DAG





$m = 16$, per row	states	edges
row-by-row	$65{,}536$	$43{,}046{,}721$
broken profile	$1{,}048{,}576$	$\leq 3{,}145{,}728$

The state count *grows* (by a factor m) while the edge count collapses (by a factor ≈ 14). Cost is edges, not states.

Predict

Audience | Predict

How many domino tilings does the 8×8 board have?

(a) $\sim 10^4$ (b) $\sim 10^5$ (c) $\sim 10^7$ (d) $\sim 10^9$

And the board with two **opposite corners** removed?

Predict

Audience | Predict

How many domino tilings does the 8×8 board have?

(a) $\sim 10^4$ (b) $\sim 10^5$ (c) $\sim 10^7$ (d) $\sim 10^9$

And the board with two **opposite corners** removed?

The DP gives $12{,}988{,}816 = (4 \cdot 901)^2$ tilings of the 8×8 board and $\mathbf{0}$ for the mutilated board: a domino covers one black and one white cell, and opposite corners have the same colour. The DP is an *exact* certificate of the colouring argument. (Aside: for planar grids Kasteleyn's Pfaffian method counts tilings in polynomial time. DP is exponential in m but handles non-planar constraints and arbitrary local rules.)

Part 3 in one slide

	I. Change the recurrence	II. Change the order
Idea	separate i from j; reduce to a query	refine the DAG; fewer in-edges
Example	convex hull trick	broken profile
Gain	$n^2 \to n$	$3^m \to m\,2^m$ per row
Price	needs algebraic/geometric structure	more (but cheaper) states

Both are statements about the DAG: time $=$ number of edges. In both cases the recurrence's *meaning* did not change.

Beyond Programming Problems: MDPs

Where dynamic programming was invented

Bellman's original setting: sequential decisions under uncertainty. An agent observes a state, acts, receives a reward, and the environment moves stochastically.

Compare with Part 1. Three assumptions are about to break:

1. transitions are **stochastic** (edges become distributions);
2. the horizon may be **unbounded**, so the state graph has **cycles**;
3. the state space may be too large to sweep (**Go**, **robotics**, **datacentre control**).

We keep the ideas (state, optimal substructure, recurrence, reconstruction) and see what each becomes.

Definition

$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with finite $\mathcal{S}, \mathcal{A}$; transition kernel $P(s' \mid s, a)$; expected reward $r(s, a)$; discount $\gamma \in [0, 1)$.

A (stationary) policy $\pi(a \mid s)$ induces the return $G_t = \sum_{k \geq 0} \gamma^k R_{t+k+1}$ and

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right], \qquad q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right].$$

The goal: $v^\star(s) = \max_\pi v_\pi(s)$ for all s simultaneously.

The Markov property is exactly the sufficiency condition of Part 1: the future depends on the past only through S_t.

Edit distance as an MDP

Audience | Discuss

Cast the edit-distance problem as an MDP: what are $\mathcal{S}$, $\mathcal{A}$, P, r, γ?

Now suppose the diagonal action *fails* with probability ε and leaves you in place. What happens to the state graph?

Edit distance as an MDP

Audience | Discuss

Cast the edit-distance problem as an MDP: what are $\mathcal{S}$, $\mathcal{A}$, P, r, γ?

Now suppose the diagonal action *fails* with probability ε and leaves you in place. What happens to the state graph?

$\mathcal{S} = \{0..m\} \times \{0..n\}$, $\mathcal{A} = \{\text{del}, \text{ins}, \text{diag}\}$, deterministic P, $r = -\text{cost}$, $\gamma = 1$, episodic with the target absorbing. A policy is a path. With failures, (i, j) gets a self-loop with probability ε: the graph is no longer a DAG and $v(i, j)$ appears on both sides of its own equation.

The Bellman expectation equation

Since $G_t = R_{t+1} + \gamma G_{t+1}$, the tower property and the Markov property give

$$v_\pi(s) = \sum_a \pi(a \mid s) \Big[r(s,a) + \gamma \sum_{s'} P(s' \mid s, a)\, v_\pi(s') \Big].$$

In matrix form, with $P_\pi(s,s') = \sum_a \pi(a|s) P(s'|s,a)$ and r_π similarly:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi \quad \Longleftrightarrow \quad v_\pi = (I - \gamma P_\pi)^{-1} r_\pi.$$

$I - \gamma P_\pi$ is invertible since the spectral radius of γP_π is $\gamma < 1$; equivalently $v_\pi = \sum_k \gamma^k P_\pi^k r_\pi$.

A linear system of $|\mathcal{S}|$ equations: the analogue of "evaluate every state from its predecessors", but simultaneous rather than ordered.

The Bellman optimality equation

$$v^\star(s) = \max_a \Big[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, v^\star(s') \Big]$$

$$q^\star(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} q^\star(s',a').$$

Any policy greedy with respect to $v^\star$, $\pi^\star(s) \in \arg\max_a \big[r(s,a) + \gamma \sum_{s'} P(s'|s,a) v^\star(s') \big]$, is optimal.

- **Optimal substructure** is now the *principle of optimality*: a tail of an optimal policy is optimal from the state where the tail starts.
- **Reconstruction** (step 4) is the $\arg\max$: we recover the policy from values.
- Deterministic transitions and $\gamma = 1$ give back $d(v) = \min\{d(u) + w\}$ (with $\max \leftrightarrow \min$).

Finite horizon: Part 1 verbatim

Horizon T, terminal reward $r_T(s)$. State space (t, s) is a DAG (time increases).

$$V_T(s) = r_T(s) \qquad \text{base case}$$

$$V_t(s) = \max_a \Big[r_t(s,a) + \sum_{s'} P_t(s'|s,a)\, V_{t+1}(s') \Big] \qquad \text{inductive step}$$

Evaluate for $t = T-1, \ldots, 0$ (bottom-up); the optimal policy is the argmax (reconstruction). Cost: $\Theta(T\,|\mathcal{S}|\,|\mathcal{A}|\,b)$ with branching factor $b \le |\mathcal{S}|$.

The only new feature: the dependency structure is an AND/OR graph. A max node (we *choose*) feeds an expectation node (nature *chooses*). This is backward induction, i.e. the recipe of Part 1 with a different semiring-like composition.

Infinite horizon: the DAG disappears

With $T = \infty$ and a stationary problem, the equations for $v^\star$ are cyclic: there is no base case and no topological order. Define the Bellman operator

$$(\mathcal{T}V)(s) = \max_a \Big[r(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \Big].$$

Theorem

$\mathcal{T}$ is a γ-contraction in $\|\cdot\|_\infty$. Hence $v^\star$ is its unique fixed point and $\|\mathcal{T}^k V_0 - v^\star\|_\infty \le \gamma^k \|V_0 - v^\star\|_\infty$ for every V_0.

Proof.

Use $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$:

$$|(\mathcal{T}V)(s) - (\mathcal{T}U)(s)| \le \gamma \max_a \sum_{s'} P(s'|s,a)\, |V(s') - U(s')| \le \gamma \|V - U\|_\infty . \qquad \square$$

What replaces each ingredient?

Audience | Discuss

What is the counterpart, in the cyclic setting, of each element of the DP recipe?

Recipe element	DAG (Part 1)	Cyclic MDP
Base case	$d(s) = 0$	
Evaluation order	topological order	
Complexity	$\lvert V \rvert \times$ cost per state	
Reconstruction	argmin predecessor	

What replaces each ingredient?

Audience | Discuss

What is the counterpart, in the cyclic setting, of each element of the DP recipe?

Recipe element	DAG (Part 1)	Cyclic MDP
Base case	$d(s) = 0$	none: contraction gives a unique fixed point; any V_0
Evaluation order	topological order	sweeps $V_{k+1} = \mathcal{T}V_k$; convergence only in the limit
Complexity	$\lvert V\rvert\times$ cost per state	(cost of a sweep) $\times\, O\big(\frac{\log(1/\epsilon)}{1-\gamma}\big)$ sweeps
Reconstruction	argmin predecessor	greedy policy w.r.t. V; loss $\leq \frac{2\gamma}{1-\gamma}\,\lVert V - v^\star\rVert_\infty$

Value iteration

$$V_{k+1} = \mathcal{T} V_k, \qquad V_0 \text{ arbitrary}.$$

One sweep costs $\Theta(|\mathcal{S}|\,|\mathcal{A}|\,b)$. A computable error certificate follows from the contraction property:

$$\|V_{k+1} - v^\star\|_\infty \le \tfrac{\gamma}{1-\gamma}\,\|V_{k+1} - V_k\|_\infty .$$

Proof.
$\|V_{k+1} - v^\star\|_\infty \le \|V_{k+1} - \mathcal{T}V_{k+1}\|_\infty + \|\mathcal{T}V_{k+1} - \mathcal{T}v^\star\|_\infty \le$
$\gamma\,\|V_k - V_{k+1}\|_\infty + \gamma\,\|V_{k+1} - v^\star\|_\infty$. □

For the greedy policy π_V one has $\|v_{\pi_V} - v^\star\|_\infty \le \frac{2}{1-\gamma}\,\|\mathcal{T}V - V\|_\infty$.

Observation: the *policy* stabilises long before the *values* do.

A worked example

$\gamma = 0.9$. State s_1: action a: $r = 1$, stay in s_1; action b: $r = 0$, go to s_2 w.p. 0.8, else stay. State s_2: action c: $r = 2$, stay; action d: $r = 3$, go to s_1.

k	$V_k(s_1)$	$V_k(s_2)$	greedy at s_1
0	0	0	a
1	1	3	?
2	2.340	4.700	b
3	3.805	6.230	b
4	5.171	7.607	b
∞	17.561	20	b

Audience | Predict

From V_1, which action does the greedy policy choose at s_1?

A worked example

$\gamma = 0.9$. State s_1: action a: $r = 1$, stay in s_1; action b: $r = 0$, go to s_2 w.p. 0.8, else stay. State s_2: action c: $r = 2$, stay; action d: $r = 3$, go to s_1.

k	$V_k(s_1)$	$V_k(s_2)$	greedy at s_1
0	0	0	a
1	1	3	b
2	2.340	4.700	b
3	3.805	6.230	b
4	5.171	7.607	b
∞	17.561	20	b

Audience | Predict

From V_1, which action does the greedy policy choose at s_1?

$q(s_1, a) = 1 + 0.9 \cdot 1 = 1.9$,
$q(s_1, b) = 0.9(0.8 \cdot 3 + 0.2 \cdot 1) = 2.34$.

Policy iteration

Alternate evaluation (solve $v_\pi = (I - \gamma P_\pi)^{-1} r_\pi$) and improvement ($\pi'(s) \in \arg\max_a q_\pi(s,a)$).

Theorem (policy improvement)

$v_{\pi'} \geq v_\pi$ pointwise, strictly somewhere unless π is optimal.

Sketch.

$\mathcal{T}_{\pi'} v_\pi = \mathcal{T} v_\pi \geq v_\pi$ and $\mathcal{T}_{\pi'}$ is monotone, so $v_\pi \leq \mathcal{T}_{\pi'} v_\pi \leq \mathcal{T}_{\pi'}^2 v_\pi \leq \cdots \to v_{\pi'}$. □

Finitely many policies and strict improvement: termination; in practice only a handful of iterations. Ye (2011): strongly polynomial for fixed γ.

In the example, starting from (a, c): $v = (10, 20) \Rightarrow$ improve to $(b, c) \Rightarrow v = (17.561, 20)$, stable after two iterations.

Asynchronous dynamic programming

Value iteration is *synchronous*: V_{k+1} is computed entirely from V_k (Jacobi), and every sweep visits every state.

Assumptions we are about to drop:

1. a global sweep that touches all $|\mathcal{S}|$ states each iteration;
2. a fixed schedule and a barrier between iterations (a global clock for distributed computation);
3. all states are equally worth updating.

Asynchronous DP. At each step choose any state s_t and set

$$V(s_t) \leftarrow (\mathcal{T}V)(s_t),$$

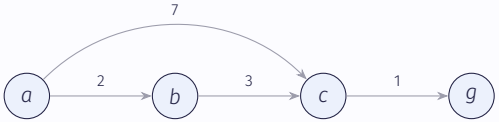
using whatever values of the other states are currently available. Order, frequency and even staleness of reads are free.

Schedules for asynchronous updates

- **In place (Gauss–Seidel):** use updated values immediately; typically needs fewer sweeps.
- **Prioritised sweeping:** update the state with the largest Bellman error $|(\mathcal{T}V)(s) - V(s)|$; when $V(s')$ changes, enqueue its predecessors (Moore–Atkeson 1993).
- **Real-time DP (RTDP):** update only along trajectories of the greedy policy from the start states (Barto–Bradtke–Singh 1995). With an optimistic initialisation it converges on the states reachable under optimal policies and never touches the rest.
- **Heuristic search:** A*/LAO* are RTDP-like schedules on AND/OR graphs.

Dijkstra is asynchronous value iteration with the schedule "finalise the state with the smallest tentative value": each state is updated once. Bellman–Ford is the synchronous schedule.

Choose your schedule

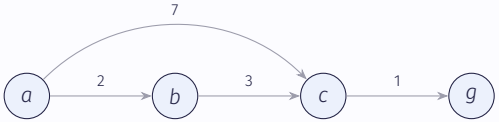


$V(g) = 0$, all other $V = \infty$; update $V(s) \leftarrow \min_{s \to s'} \{w + V(s')\}$.

Audience | Predict

1. How many synchronous sweeps until $V(a)$ is final?
2. Which update order finishes in one pass?

Choose your schedule



$V(g) = 0$, all other $V = \infty$; update $V(s) \leftarrow \min_{s \to s'} \{w + V(s')\}$.

Audience | Predict

1. How many synchronous sweeps until $V(a)$ is final?
2. Which update order finishes in one pass?

(1) Three sweeps: information travels one edge per sweep. (2) c, b, a: a reverse topological order. The bottom-up evaluation of Part 1 is the best possible asynchronous schedule when the graph is a DAG; for cyclic graphs, Dijkstra's priority queue computes such an order on the fly.

Why asynchronous updates converge

Theorem (Bertsekas 1982; Bertsekas–Tsitsiklis 1989)

Let $\mathcal{T}$ be a γ-contraction in $\|\cdot\|_\infty$. If each coordinate is updated infinitely often, and stale information is eventually flushed, then $V_t \to v^\star$ for any initial V_0.

Box argument.

Let $D = \|V_0 - v^\star\|_\infty$ and $B_k = \{V : \|V - v^\star\|_\infty \le \gamma^k D\}$. Let t_k be a time after which every coordinate has been updated at least once using only information from after t_{k-1}. Claim: $V_t \in B_k$ for $t \ge t_k$. Induction: an update of coordinate s reads a vector in B_{k-1}, so $|V_{t+1}(s) - v^\star(s)| = |(\mathcal{T}V)(s) - (\mathcal{T}v^\star)(s)| \le \gamma \cdot \gamma^{k-1} D$, and further updates keep s within the box. □

The per-coordinate contraction is all we need: no global clock, order, or synchronisation.

From asynchronous DP to learning

Replace the expectation over P by a *sample* (s, a, r, s') and take a step of size α_t:

$$Q(s,a) \leftarrow Q(s,a) + \alpha_t \Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \Big].$$

This is Q-learning: asynchronous value iteration on $q^\star$ with stochastic approximation (Watkins–Dayan 1992; Tsitsiklis 1994). It converges when every pair (s,a) is visited infinitely often and $\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$.

Carries over	Fixed point of the Bellman operator; contraction; state sufficiency; greedy extraction of the policy
Changes	Model P unknown (sampling replaces expectation); updates asynchronous by construction
Breaks	With function approximation the composed operator need not be a contraction (Baird's counterexample)

Discussion

Audience | Discuss

A state space with 10^{20} states (a board game, a robot with continuous sensors).

1. Which of the Part 1 requirements do you give up first?
2. What replaces the table $V : \mathcal{S} \to \mathbb{R}$?
3. Which guarantee from the contraction proof do you lose?

Other Places, and What We Learned

One DAG, many semirings

The recurrence $d(v) = \bigoplus_{u \to v} d(u) \otimes w(u, v)$ is a path problem over a semiring $(K, \oplus, \otimes)$. Changing the semiring changes the problem, not the algorithm.

$(\oplus, \otimes)$	Computes	Examples
$(\min, +)$	shortest path	edit distance, Part 1
$(\max, +)$	longest path	Viterbi in log-space, critical path
$(+, \times)$	weighted path sum	domino tilings (Part 3), HMM forward algorithm
$(\max, \times)$	most probable path	Viterbi, probabilistic CYK
$(\min, \max)$	bottleneck path	Case 1 (split array), minimax routing
$(\vee, \wedge)$	reachability	parsing recognition, regular-expression matching

Name the semiring

Audience | Vote

Which semiring (if any) governs each recurrence?

(a) $N(i) = \sum_{j<i} N(j)\,[\text{edge } j \to i\,]$ (number of paths)

(b) $f(i,j) = \min_p \max\{f(p, j-1), S_i - S_p\}$

(c) $V_t(s) = \max_a \left[r + \sum_{s'} P(s'|s,a) V_{t+1}(s')\right]$

Name the semiring

Audience | Vote

Which semiring (if any) governs each recurrence?

(a) $N(i) = \sum_{j<i} N(j)\,[\text{edge } j \to i\,]$ (number of paths)

(b) $f(i,j) = \min_p \max\{f(p, j-1), S_i - S_p\}$

(c) $V_t(s) = \max_a \left[r + \sum_{s'} P(s'|s,a) V_{t+1}(s')\right]$

(a) $(+,\times)$. (b) $(\min,\max)$. (c) None: it alternates $\max$ over actions with an expectation over successors, so it is a path problem on an AND/OR graph (a hypergraph). For a *fixed* policy it collapses to $(+,\times)$ and becomes linear: $v_\pi = r_\pi + \gamma P_\pi v_\pi$.

Linear optimisation: a staircase LP

Production planning. Periods $t = 1, \dots, T$; demand d_t, unit production cost c_t, capacity u, unit holding cost h, inventory s_t with s_1 given.

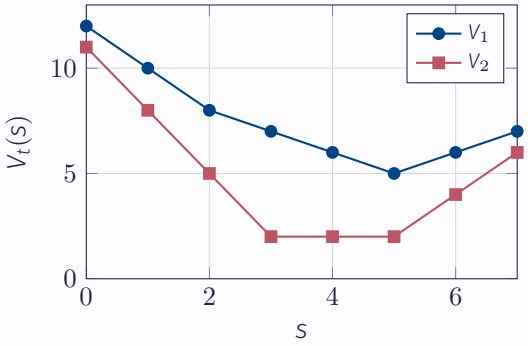
$$\begin{aligned} \min \quad & \sum_{t=1}^{T} \big(c_t x_t + h\, s_{t+1} \big) \\ \text{s.t.} \quad & s_{t+1} = s_t + x_t - d_t, \qquad 0 \le x_t \le u, \qquad s_{t+1} \ge 0. \end{aligned}$$

This is an LP in $2T$ variables. Constraint t involves only (s_t, x_t, s_{t+1}): a staircase structure. Inventory s_t is a sufficient state, and the LP is a path in a (continuous) state space:

$$V_{T+1} \equiv 0, \qquad V_t(s) = \min_{\substack{0 \le x \le u \\ s + x - d_t \ge 0}} \Big\{ c_t x + h(s + x - d_t) + V_{t+1}(s + x - d_t) \Big\}.$$

The state is real-valued: no table. What does V_t look like?

The value functions are piecewise linear and convex



$T = 3$, $d = (2, 3, 2)$, $c = (1, 3, 1)$, $u = 3$, $h = 1$.

Theorem

Each V_t is convex and piecewise linear on its domain.

Proof. $V_t(s)$ is the optimal value of an LP whose right-hand side depends affinely on s; partial minimisation of a jointly convex polyhedral function over a polyhedron is convex and polyhedral. By induction, V_{t+1} has the same form.

DP is exact here without discretisation: represent V_t by its breakpoints or its supporting lines (*cuts*).

Reading the slopes

Audience | Predict

V_2 has slope -3 on $[0, 3]$, slope 0 on $[3, 5]$ and slope $+2$ on $[5, 7]$. Explain each slope in terms of what one extra unit of inventory does.

Reading the slopes

Audience | Predict

V_2 has slope -3 on $[0, 3]$, slope 0 on $[3, 5]$ and slope $+2$ on $[5, 7]$. Explain each slope in terms of what one extra unit of inventory does.

- $[0, 3]$: one extra unit avoids producing a unit at $c_2 = 3$ in period 2: saving 3.
- $[3, 5]$: demand of period 2 is covered; the extra unit then replaces production at $c_3 = 1$ in period 3 but costs holding $h = 1$: net 0.
- $[5, 7]$: all demand is covered; the extra unit is just held through periods 2 and 3: cost $2h = 2$.

The slopes of V_t are the LP dual variables (shadow prices). Stochastic dual dynamic programming (SDDP) builds V_t from exactly these supporting lines.

The other direction: MDPs as linear programs

For $\mu > 0$ consider

$$\min_v \sum_s \mu(s)v(s) \quad \text{s.t.} \quad v(s) \geq r(s,a) + \gamma \sum_{s'} P(s'|s,a)v(s') \quad \forall s, a.$$

Claim: $v^\star$ is the unique optimum. Feasibility means $v \geq \mathcal{T}v$; by monotonicity of $\mathcal{T}$, $v \geq \mathcal{T}^k v \to v^\star$. So every feasible v dominates $v^\star$, which is itself feasible.

The dual variables $x(s,a) \geq 0$ are discounted state-action occupancy measures:

$$\max_x \sum_{s,a} r(s,a)x(s,a) \quad \text{s.t.} \quad \sum_a x(s',a) - \gamma \sum_{s,a} P(s'|s,a)x(s,a) = \mu(s').$$

Complementary slackness says $x(s,a) > 0$ only if a attains the maximum in the Bellman optimality equation. **Bellman optimality = LP optimality conditions.**

Dynamic programming elsewhere

Area	Dynamic programming in disguise
Computational biology	Needleman–Wunsch / Smith–Waterman alignment; RNA folding (Nussinov, Zuker)
Language and speech	Viterbi decoding; CYK; inside–outside algorithm
Probabilistic inference	Variable elimination / junction trees: DP over a tree decomposition of the model
Optimal control	LQR: $V_t(x) = x^\top P_t x$ with the Riccati recursion for P_t
Operations research	Multi-stage (stochastic) LPs and SDDP; DP-based algorithms for integer programs with small coefficients
Economics	Recursive formulations of consumption and growth (Bellman equations over continuous states)

Your turn

Audience | Pair

Turn to your neighbour for three minutes. Pick a problem you work on.

1. What is a **sufficient state**? Is the state graph a DAG?
2. Which **semiring** is it? What is the **edge count**?
3. Which **lever** (recurrence, order, schedule) could reduce it?

Then we collect two or three examples, including the ones that *failed* the test.

What we learned

1. **DP is path-finding in a state space.** A sufficient state and an acyclic order give a recurrence, and complexity is the number of edges.
2. **Not every sequence of decisions is a DP.** Check for a compact sufficient state *and* for cheaper structure (monotone decision problems, exchange arguments).
3. **Speed comes from the shape of the DAG.** Rewrite the recurrence (convex hull trick) or refine the order (broken profile).
4. **Beyond DAGs, DP is a fixed-point computation.** Bellman operators are contractions; value iteration, policy iteration and *asynchronous* schedules are different ways of reaching the fixed point.
5. **The reach is wide.** Semirings, linear programs, control and learning share the same recurrence.

Questions and discussion

References (1/2)

- R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- D. Bertsekas. Distributed dynamic programming. *IEEE Trans. Automatic Control*, 1982.
- D. Bertsekas, J. Tsitsiklis. *Parallel and Distributed Computation*. Prentice Hall, 1989.
- A. Barto, S. Bradtke, S. Singh. Learning to act using real-time dynamic programming. *Artificial Intelligence*, 1995.
- R. Sutton, A. Barto. *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018 (Ch. 3–4).
- M. Puterman. *Markov Decision Processes*. Wiley, 1994.
- Y. Ye. The simplex and policy-iteration methods are strongly polynomial for the Markov decision problem with a fixed discount rate. *Math. of Operations Research*, 2011.
- A. Moore, C. Atkeson. Prioritized sweeping. *Machine Learning*, 1993.

References (2/2)

- C. Watkins, P. Dayan. Q-learning. *Machine Learning*, 1992; J. Tsitsiklis. Asynchronous stochastic approximation and Q-learning. *Machine Learning*, 1994.
- A. Backurs, P. Indyk. Edit distance cannot be computed in strongly subquadratic time (unless SETH is false). STOC 2015.
- E. Myers. An $O(ND)$ difference algorithm and its variations. *Algorithmica*, 1986.
- D. Hirschberg. A linear space algorithm for computing maximal common subsequences. *CACM*, 1975.
- M. Pereira, L. Pinto. Stochastic dual dynamic programming. *Math. Programming*, 1991.
- R. Larmore, B. Schieber. On-line dynamic programming with applications to the prediction of RNA secondary structure. *J. Algorithms*, 1991.

Backup: the Damerau–Levenshtein state

The unrestricted distance (Lowrance–Wagner) allows material to be edited between the two transposed symbols. For each (i, j) let $i' < i$ be the last position with $x_{i'} = y_j$ and $j' < j$ the last position with $y_{j'} = x_i$ (when both exist). Add the edge

$$(i' - 1, j' - 1) \to (i, j), \qquad w = (i - i' - 1) + 1 + (j - j' - 1),$$

meaning: delete the gap in x, transpose, insert the gap in y. Time $O(mn)$ with $O(|\Sigma|)$ extra memory for the last-occurrence tables. The state is still (i, j), but the *edge set* reaches back arbitrarily far.